

Replicating Neural Scaling Laws: A Decoder-Only Transformer Study

Parth Grover
CSOC'26 – NLP Track Capstone Project

June 29, 2026

Abstract

We present a replication study of the empirical scaling laws for neural language models introduced by Kaplan et al. (2020). A decoder-only autoregressive Transformer was implemented from foundational tensor operations in PyTorch, without reliance on high-level abstractions such as `nn.MultiheadAttention`. Using Tiny Shakespeare dataset, we conduct two controlled sweeps: a parameter scaling sweep across three model sizes (Tiny, Small, Medium) and a data scaling sweep across four token budgets (10%, 25%, 50%, 100%) using the Small model. We fit power-law exponents α_N and α_D in log-log space and obtain $\alpha_N = 0.066$, $\alpha_D = 0.123$, yielding a scaling ratio $\gamma = \alpha_N/\alpha_D \approx 0.538$. We compare this to OpenAI’s reported $\gamma \approx 0.80$ and discuss the implications.

1 Introduction

Modern Natural Language Processing relies on the empirical observation that language modeling performance improves predictably as a power law with respect to model parameters N and dataset size D . This report documents a controlled replication of this finding at small scale, suitable for execution on free-tier compute (Google Colab / Kaggle).

The study is divided into two levels, mirroring the project structure:

- **Level 1:** Implementation of a decoder-only Transformer architecture from scratch, including causal multi-head attention, position-wise feed-forward layers, pre-layer normalization, positional encodings, and a weight-tied language modeling head.
- **Level 2:** Two controlled sweeps – one varying model capacity (N) at fixed data and step count, and one varying data budget (D) at fixed model capacity – used to empirically fit α_N and α_D .

2 Mathematical Formalism

This section defines the exact mathematical components implemented at Level 1.

2.1 Causal Multi-Head Attention

Given an input $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$, queries, keys, and values are obtained via learned linear projections and split across H heads of dimension $d_k = d_{\text{model}}/H$. Attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (1)$$

where the causal mask $M \in \mathbb{R}^{T \times T}$ is defined as:

$$M_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases} \quad (2)$$

This mask ensures that position i can only attend to positions $j \leq i$, preserving the autoregressive property required for causal language modeling.

2.2 Position-wise Feed-Forward Network

Each Transformer block contains a two-layer MLP applied identically at every sequence position:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (3)$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, and $d_{\text{ff}} = 4 \times d_{\text{model}}$.

2.3 Pre-Layer Normalization

To stabilize gradient flow as depth increases, Layer Normalization is applied on the input branch of each sublayer (Pre-LN), with residual connections:

$$x^{(k+1)} = x^{(k)} + \text{SubLayer} \left(\text{LN}(x^{(k)}) \right) \quad (4)$$

2.4 Positional Encoding

Sequential order is injected via sinusoidal positional embeddings positional encodings, added to the token embeddings prior to the first Transformer block.

2.5 Weight-Tied Language Modeling Head

The output projection back to vocabulary space reuses the token embedding matrix $E \in \mathbb{R}^{V_{\text{vocab}} \times d_{\text{model}}}$:

$$\text{logits} = h E^T \quad (5)$$

This weight-tying reduces the total parameter count and is excluded from the non-embedding parameter calculation.

2.6 Non-Embedding Parameter Count

Following the methodology specified for this study, embedding parameters are excluded from the model-size variable N used in the scaling law fits, since they scale with vocabulary size rather than network expressivity. For a network with L layers and hidden dimension d_{model} :

$$N \approx L \times (12 \cdot d_{\text{model}}^2 + 4 \cdot d_{\text{model}}) \quad (6)$$

This was validated programmatically by summing parameters across all named modules excluding those containing 'embed' or 'lm_head' in their parameter names.

3 Replication Methodology

3.1 Dataset

We have used the **Tiny Shakespeare** corpus, a character-level dataset of approximately 1.1MB . Characters are mapped to integers via a direct dictionary lookup.

A standard 90/10 train/validation split was applied to the full token stream. For the data scaling sweep, the training set was further truncated to 10%, 25%, 50%, and 100% of its original length, while the validation set remained fixed across all runs.

3.2 Optimization Configuration

All sweeps were run with identical optimizer hyperparameters to isolate the effect of the swept variable:

Hyperparameter	Value
Optimizer	Adam
Learning Rate	0.0005
Batch Size	128
Training Steps	3,000
Loss Function	Cross-Entropy

Table 1: Optimization configuration held constant across sweeps.

3.3 Parameter Scaling Sweep

Three model configurations (Tiny, Small, Medium) were trained for exactly 3,000 steps on the full (100%) dataset. Architecture details and measured non-embedding parameter counts are reported in Table 3.

Scale	Layers (L)	d_{model}	Heads (H)	Context (T)	N (measured)
Tiny	2	64	2	128	90752
Small	4	128	4	256	723456
Medium	6	256	8	256	4333056

Table 2: Measured non-embedding parameter counts for the parameter scaling sweep.

3.4 Data Scaling Sweep

The Small model configuration was held fixed while the training data budget was varied across four token counts, reported in Table 3.

Budget	% of Full Dataset	Token Count (D)
D_{10}	10%	100385
D_{25}	25%	250963
D_{50}	50%	501927
D_{100}	100%	1003854

Table 3: Token budgets used in the data scaling sweep (Small model, fixed architecture).

4 Results

4.1 Training and Validation Loss Curves

Figures 1 and 2 show training and validation loss trajectories for the parameter scaling sweep (Tiny, Small_100, Medium). Figures 3 and 4 show the same for the data scaling sweep (Small_10, Small_25, Small_50; Small_100 is shared with the parameter sweep above).

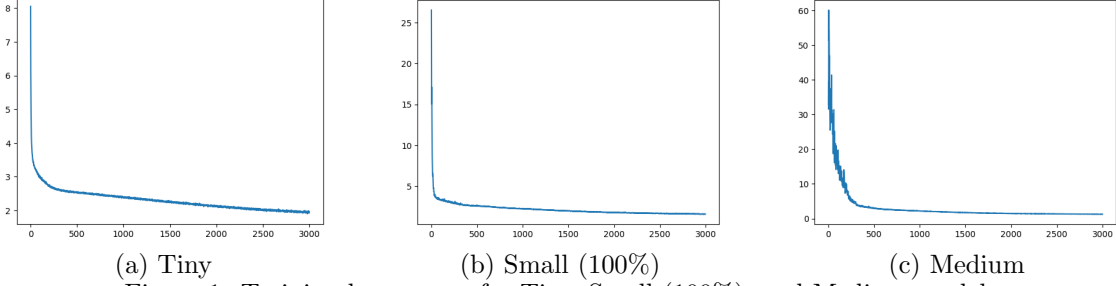


Figure 1: Training loss curves for Tiny, Small (100%), and Medium models.

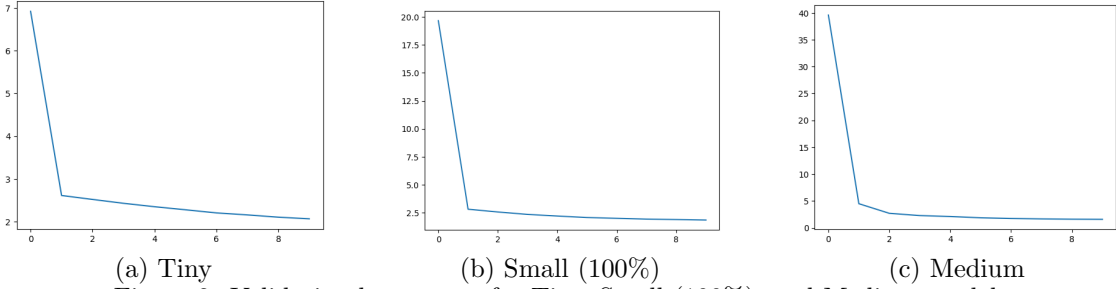


Figure 2: Validation loss curves for Tiny, Small (100%), and Medium models.

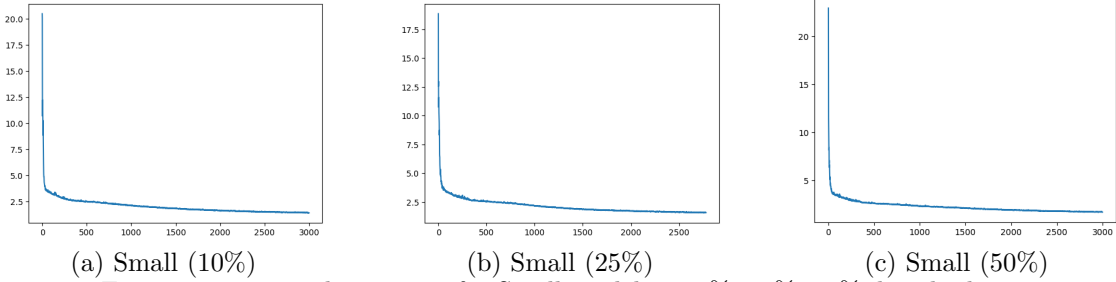


Figure 3: Training loss curves for Small model at 10%, 25%, 50% data budgets.

4.2 Best Validation Loss Summary

Table 4 summarizes the best (minimum) validation loss achieved by each model/budget configuration across all six training runs.

4.3 Power-Law Fits

Power laws were fit in log-log space via least-squares regression (`numpy.polyfit`), following:

$$\log L(N) = \text{const} - \alpha_N \log N, \quad \log L(D) = \text{const} - \alpha_D \log D \quad (7)$$

The fitted exponents and resulting scaling ratio are:

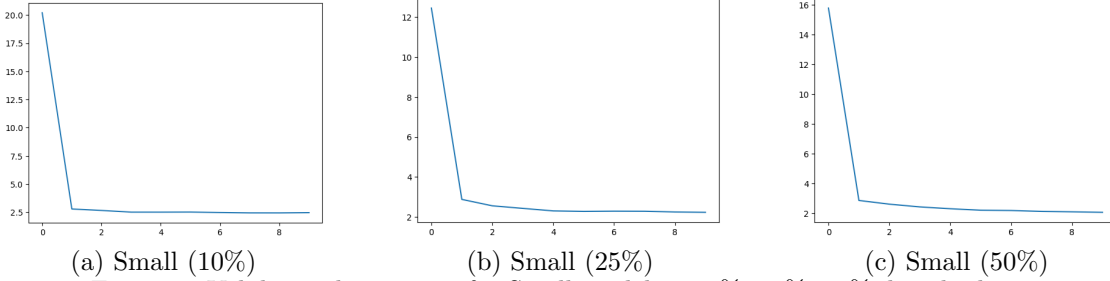


Figure 4: Validation loss curves for Small model at 10%, 25%, 50% data budgets.

Run	Variable Swept	Best Validation Loss (L)
Tiny (N)	Parameter	2.06
Small_100 (N , also D_{100})	Parameter / Data	1.83
Medium (N)	Parameter	1.59
Small_10 (D_{10})	Data	2.43
Small_25 (D_{25})	Data	2.28
Small_50 (D_{50})	Data	2.08

Table 4: Best validation loss per configuration, used as input to the power-law fits below.

$$\alpha_N = 0.066 \tag{8}$$

$$\alpha_D = 0.123 \tag{9}$$

$$\gamma = \frac{\alpha_N}{\alpha_D} = 0.538 \tag{10}$$

5 Theoretical Discussion

5.1 Comparing Exponents and the Scaling Ratio

Kaplan et al. (2020) report $\alpha_N \approx 0.076$ and $\alpha_D \approx 0.095$ for large-scale language models trained on a web-text corpus, giving $\gamma \approx 0.80$. Our replication, conducted at a much smaller scale (parameter counts on the order of 10^5 – 10^6 versus the original study’s range extending to billions of parameters) on **Tiny Shakespeare**, yields $\gamma \approx \mathbf{0.538}$.

- The observed ratio $\gamma < 1$ suggests that increasing dataset size yields greater marginal improvements than increasing parameter count, consistent with the conclusions of Kaplan et al. Our observed value of α_d is more than that Kaplan’s 0.095 which indicates that increasing the amount of training data produced stronger improvements than those reported in the original paper.
- A likely explanation is that the Tiny Shakespeare dataset is several orders of magnitude smaller than the datasets used in the original study. Consequently, the models in this replication may remain data-limited, causing performance to benefit more strongly from increases in dataset size. Furthermore, the restricted training budget of 3000 optimization steps and the substantially smaller model sizes may prevent the experiments from fully reaching the scaling regime explored by Kaplan et al.

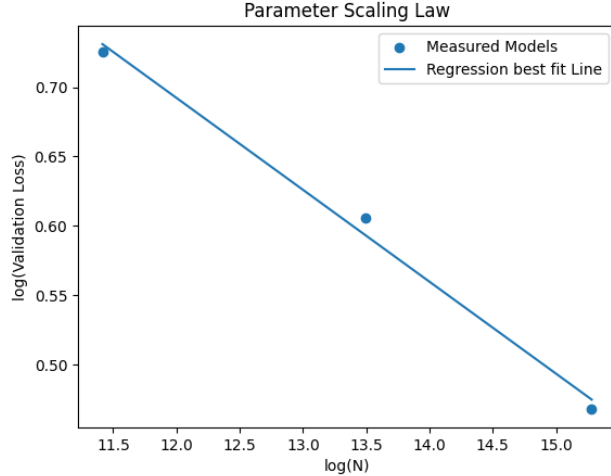


Figure 5: log-log plot of validation loss vs. non-embedding parameters N , with regression line.

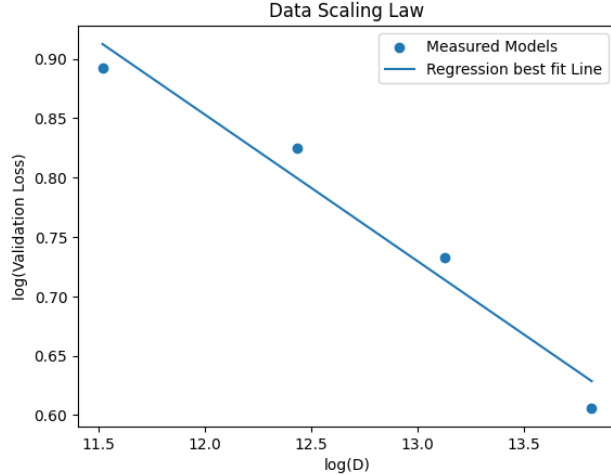


Figure 6: log-log plot of validation loss vs. data budget D , with regression line.

6 Conclusion

We built a decoder only transformer model from scratch. To verify the mathematical correctness of the implementation, the model was trained repeatedly on a single fixed batch of training data. The cross-entropy loss decreased to nearly zero, demonstrating that the network was capable of perfectly memorizing the batch. Furthermore, we replicated the empirical experiments of Kaplan et al(2020) and analyzed scaling performance across two dimensions: parameters (N) and training token dataset size (D). We used the **Tiny Shakespeare** dataset for training and evaluation and concluded that scaling the dataset yields greater marginal performance gains than scaling the parameter count, which is consistent with the findings of Kaplan et al.

References

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.